

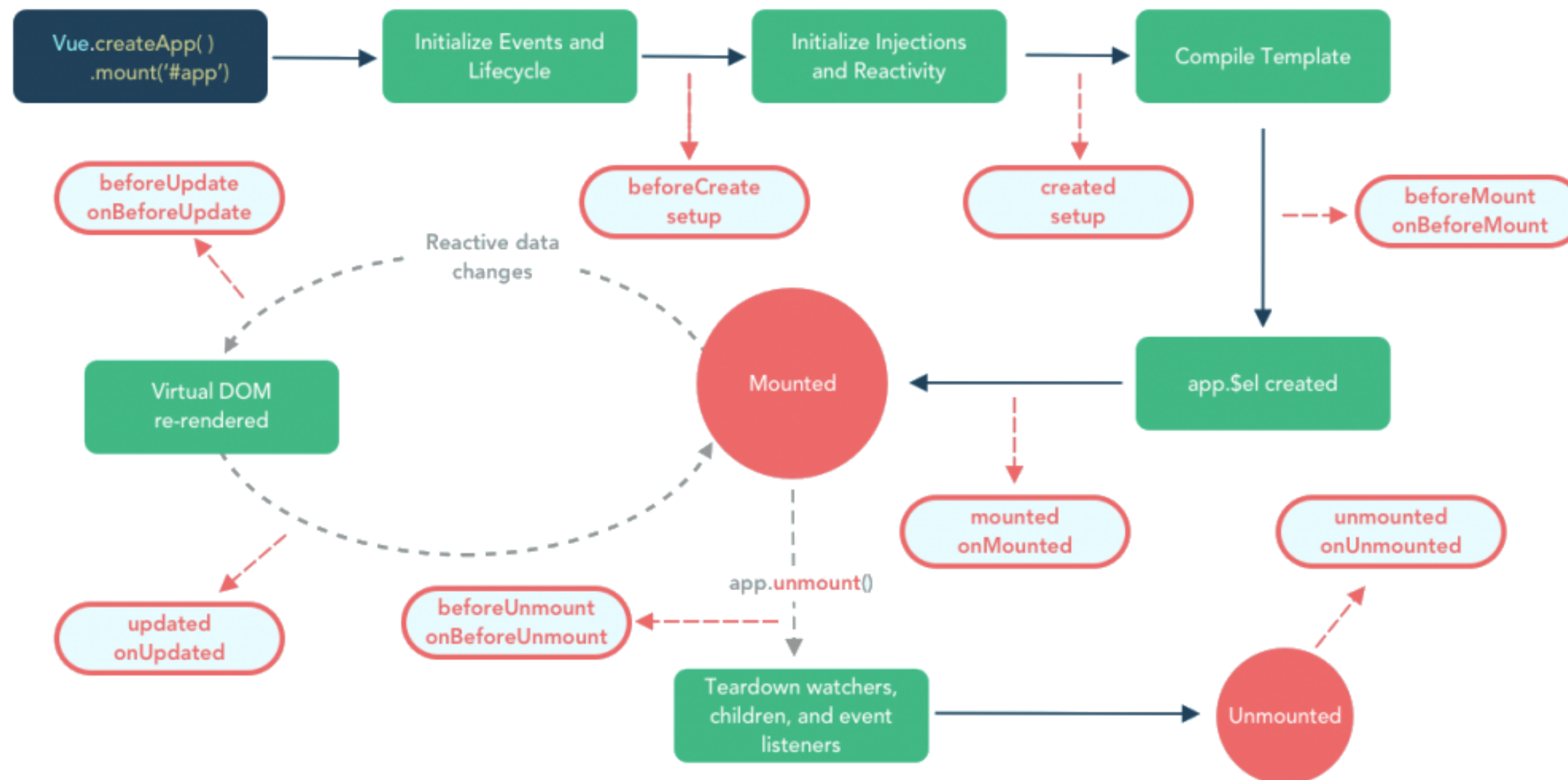
Vue.js (Components)

Interface Pessoa-Máquina - 25/26 - LEI / UM

Hugo Pacheco
hpacheco@di.uminho.pt

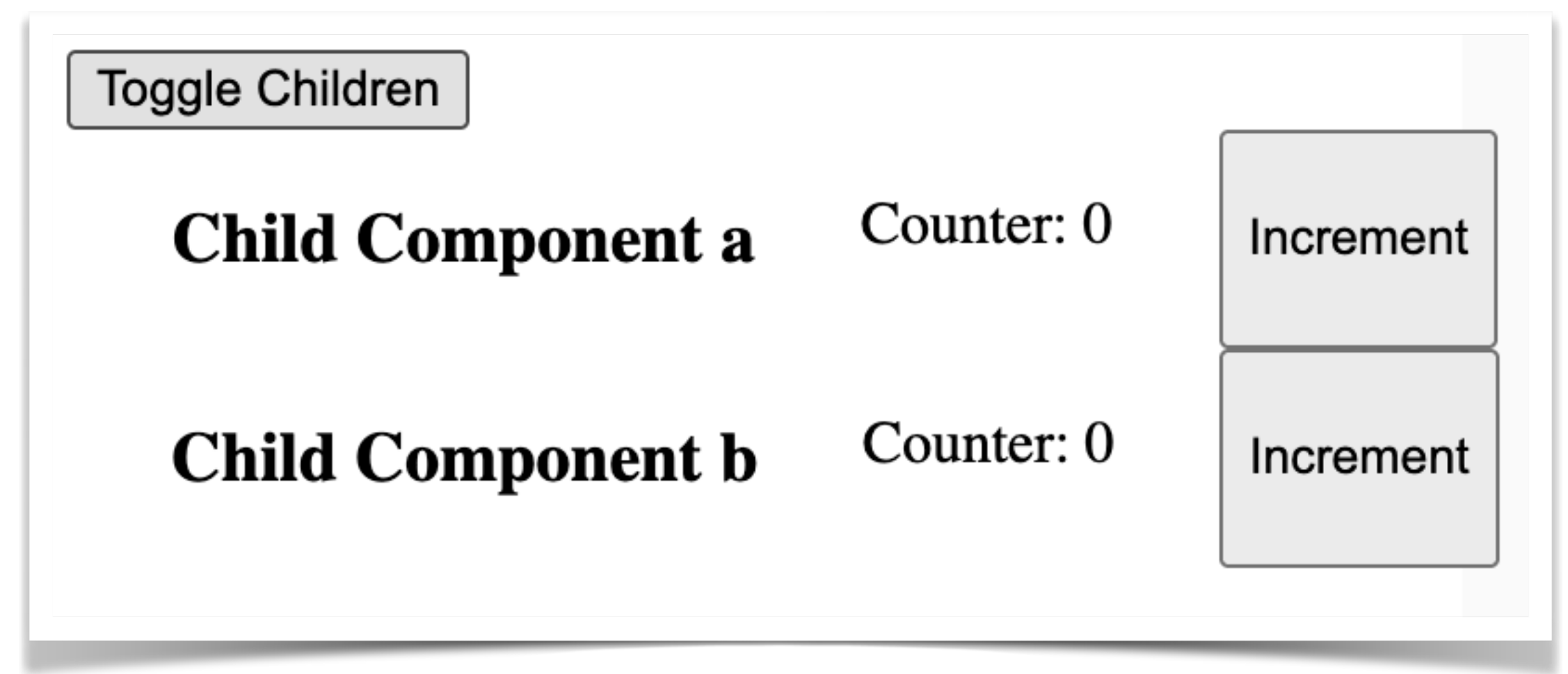
Lifecycle hooks

- Uma aplicação / componente Vue tem um ciclo de vida
- Podemos executar métodos Vue em diversas fases



Components (Exemplo 1)

- Em Vue, cada componente é declarado num ficheiro independente
- Devem-se usar multi-word names
 - Em Vue JS: `ComponentName.vue`, `import ComponentName`
 - No template HTML: `tag HTML component-name` ou `ComponentName`
- Exemplo



Components (lifecycle)

- Bastante similar a Web Components
- `<style scoped>` = Shadow CSS
- Dynamic components:
 - Por defeito, components são destruídos quando unmounted
 - 💡 Ao contrário de Web Components, em que ficam no DOM
 - Tag HTML `keep-alive` preserva o component e o seu estado
- ? `onUpdated()` $\neq$ `attributeChangedCallback(name, oldV, newV)`?
 - 💡 Virtual DOM $\Rightarrow$ não queremos saber que valores no modelo mudaram!

Components (Exemplo 1)

- Algumas notas sobre o exemplo:

💡 `keep-alive` $\neq$ `v-show`

💡 Podemos associar elementos **HTML** a refs **Vue**

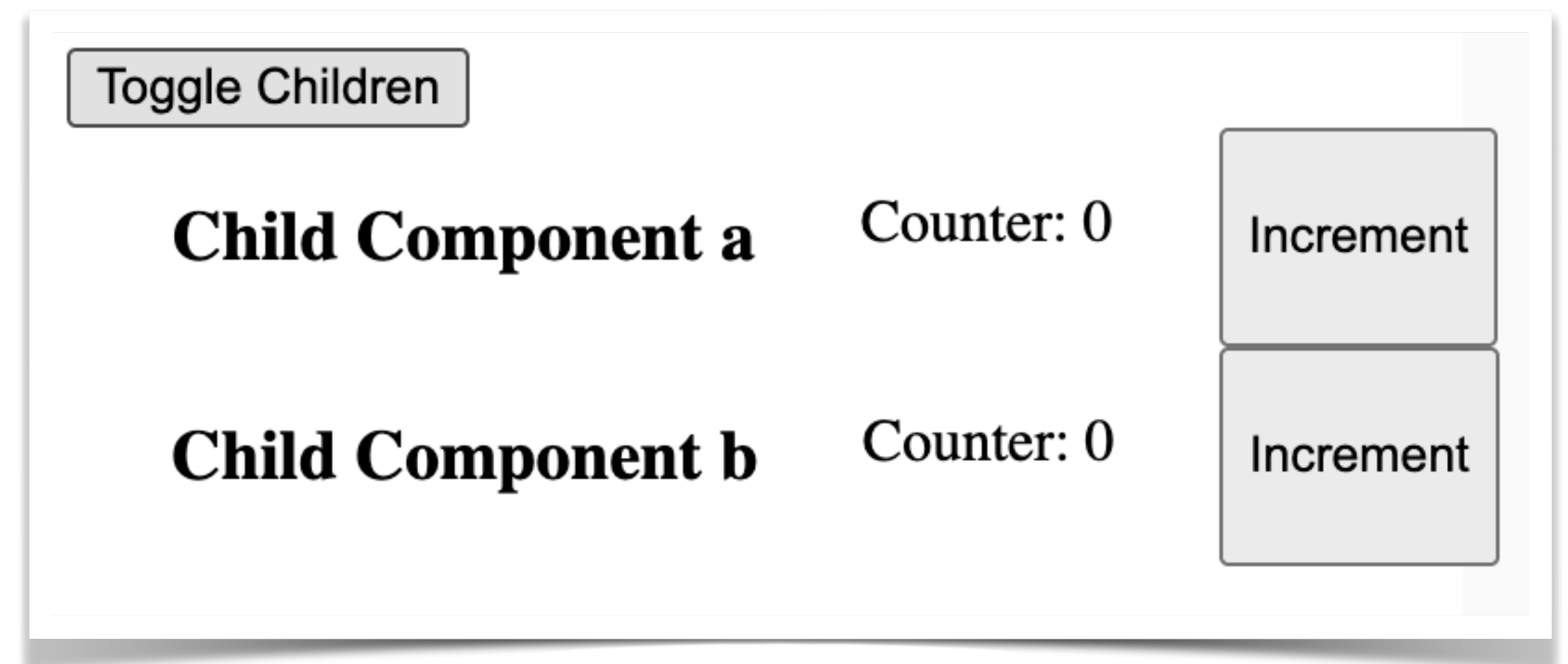
```
<div ref="root">
```

```
...
```

```
</div>
```

```
const root = ref(null);
```

```
root.value.id
```



Components (props) (Exemplo 2)

- Componentes podem receber parâmetros

```
<template>
  <h1> Child {{ name }}
</h1>
</template>
```

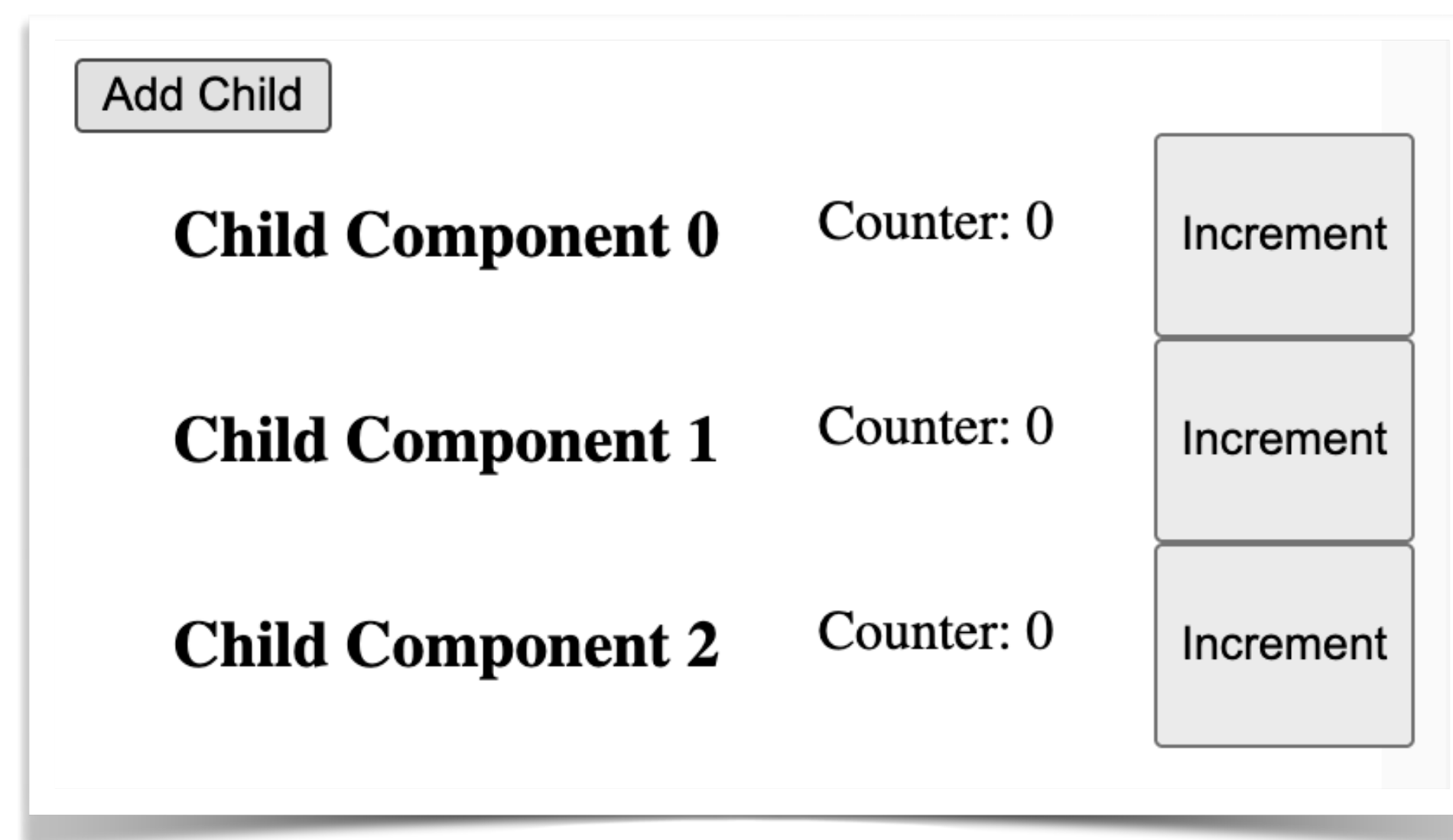
```
<script>
const props =
defineProps({
  myName: String,
});
</script>
```

Pode ter
anotações
adicionais:
type,
required,
default, ...

- Passados sob a forma de atributos HTML pelos pais

```
<child-component my-
name="{{ childName }}" />
```

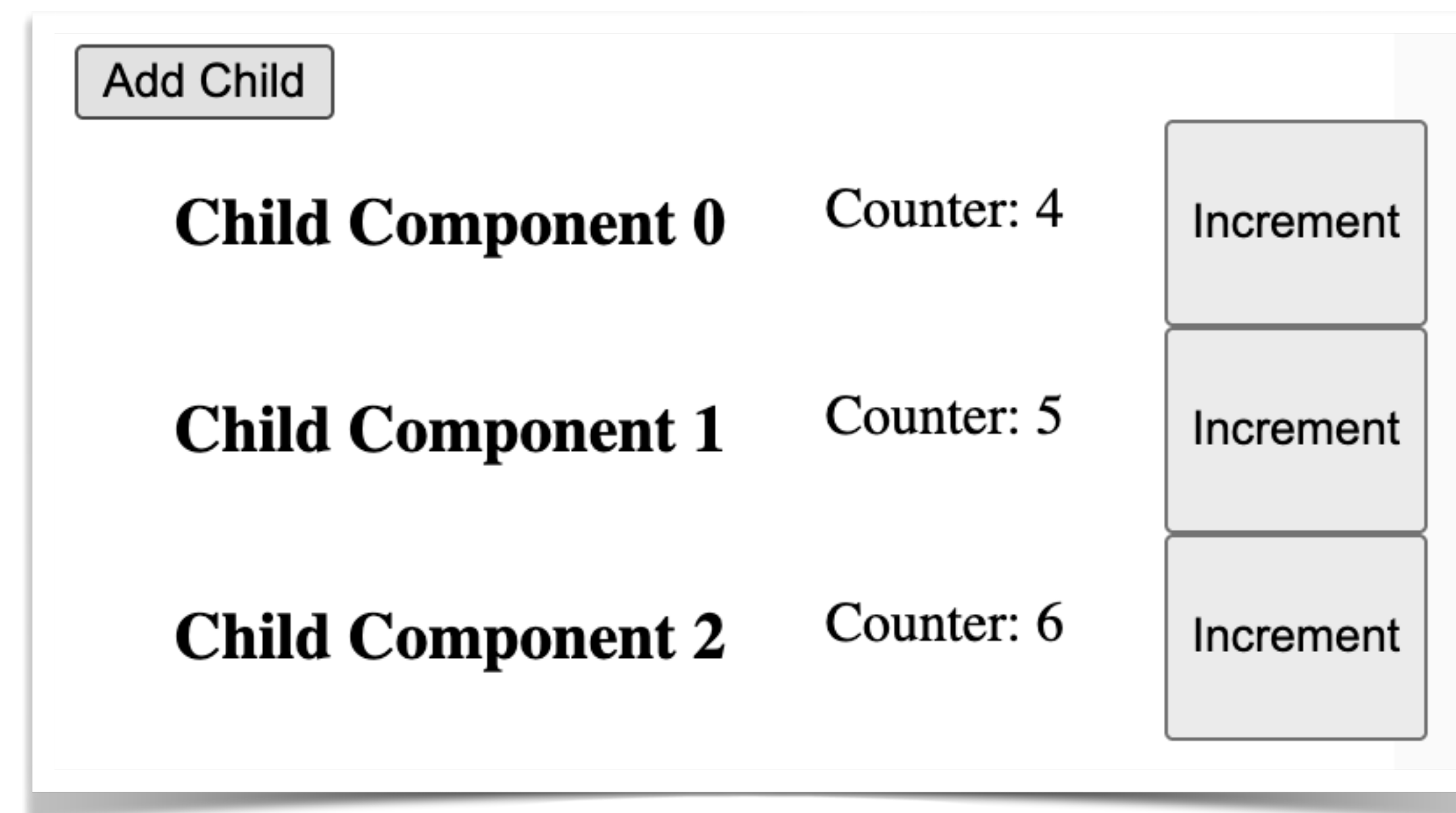
- ! Convenção de nomes diferente



Components (props) (Exemplo 3)

- Valor de uma prop é imutável
 - Ao contrário de atributos HTML em Web Components
- Vue segue uma política de **unidirectional data flow**
 - Design pattern para parâmetros mutáveis é criar uma `ref` dentro do componente filho
 - E atualizar o valor da `ref` com o da prop quando o componente é inicializado

- 💡 Para hierarquias de components, props **não** são passadas implicitamente de pais para filhos
- 💡 Comunicação direta de pai para filho

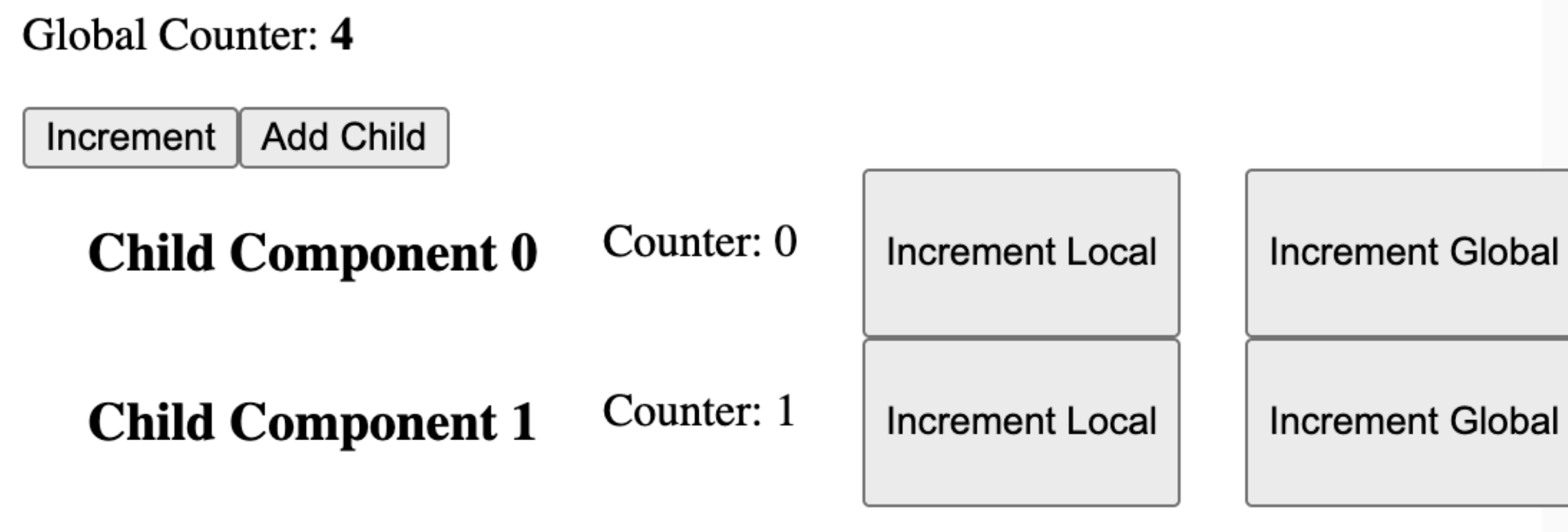


Components (props)

- ? Prop passada a child component pode depender de reactive data?
 - Sim pode, como vimos no Exemplo 3
- ? O que acontece se valor no parent component mudar?
 - Prop no child component é atualizada automaticamente
- ? O que significa então **unidirectional data flow**?
 - 💡 Child components **não podem** mutar props
 - 💡 Dados e (modificações aos dados) **podem** fluir de parent para child

Components (props) (Exemplo 4)

- Na verdade, child components podem em alguns casos mutar props
 - Valor de uma prop pode ser de qualquer tipo
- Uma forma de codificar **bidirectional data flow**
 - 💡 Podemos passar refs dentro de props
 - ! Vue pode (contra-intuitivamente) fazer unwrapping automático da ref



Components (emits) (Exemplo 5)

- Components podem emitir *custom events*

```
const emit =  
defineEmits(['inc']);  
  
emit('inc');
```

Pode ter
anotações
adicionais:
parâmetros
e tipo, etc

- ! Apenas o pai direto pode escutar o evento

```
<my-child  
@inc="increment"/>
```

- 💡 Forma tradicional de comunicar de filhos para pais
- ! Ao contrário de JS, não existe *bubbling*

Global counter: 24

Add Child

Child Component 0

Counter: 7

Increment

Child Component 1

Counter: 6

Increment

Child Component 2

Counter: 11

Increment

Components (slots) (Exemplo 6)

- Uma vantagem de components Vue é maior integração
 - Pais podem passar código Vue a components filhos

```
<card-component>
  <template #header>Child Component
  {{ child }}</template>
  <child-component />
</card-component>
```

DOM do filho para
Unnamed Slot

Template no pai
para named slot

- Filhos definem uma ou mais slots

```
<template>
  <div>
    <h2><slot name="header"></slot></h2>
    <slot></slot>
  </div>
</template>
```

Global counter: 1

Add Child

Child Component 0

Counter: 0

Increment

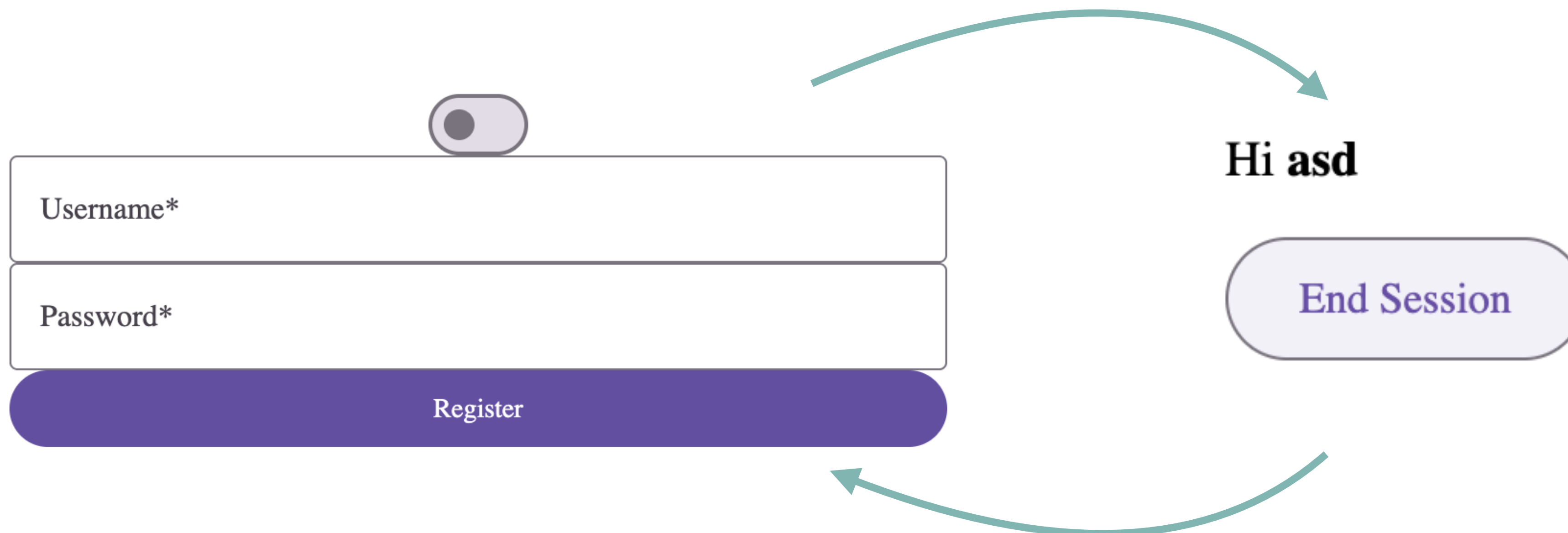
Child Component 1

Counter: 1

Increment

Exemplo (User + Session)

- Vue components são praticamente Web Components!
- Podemos migrar facilmente o exemplo anterior para Vue
- 💡 Pais podem invocar métodos de filhos
(desde que expostos pelos filhos com `defineExpose`)



Components: vs

- Remember, remember...
- Existe um preço a pagar por maior integração
- Entrevista de Tim Berners-Lee ao jornal Público
- ? **Não acha que o desenvolvimento web se tornou demasiado complexo para uma plataforma que se queria democrática?**



*“Usei algumas ferramentas mais modernas, mas desconfio da dependência excessiva que geram. [...] Se achamos a web de hoje complicada, porquê ensinar essas ferramentas aos estudantes? HTML e CSS são simples e **permitem copiar e adaptar código a partir de outra página facilmente. Essa é a beleza da web.**”*